+ Lecture No. : +

■ **Theory Content to be taught :**

1. Unit 3 : Linked list.
2. - singly linked list.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

@ Linked list :-

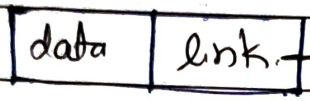
• Introduction :- A linked list is a linear ~~type~~ collection of data items called nodes, where the linear order is given by means of pointers.

In linked list, adjacency between the elements are maintained by means of links or ~~posti~~ pointers.

A link or pointer actually is the address of the subsequent element. Thus in a linked

list, data (actual content) and link (to the point to next data) both are required to be maintained.

An element in a linked list is specially term as a node and a node is divided into two parts namely data & next pointer or link.

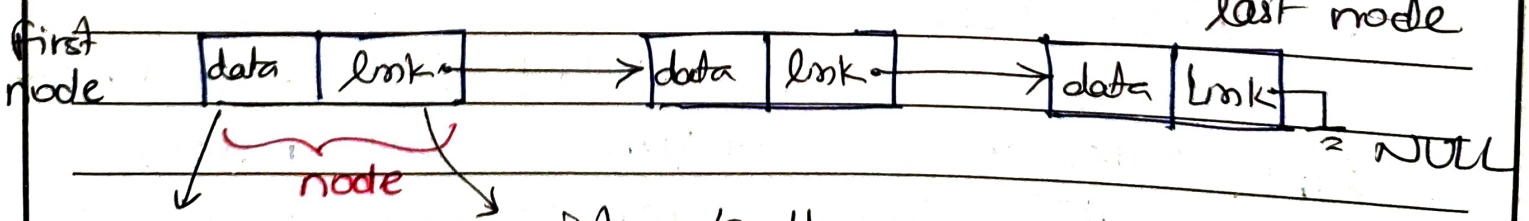


→ Link to the next pointer.

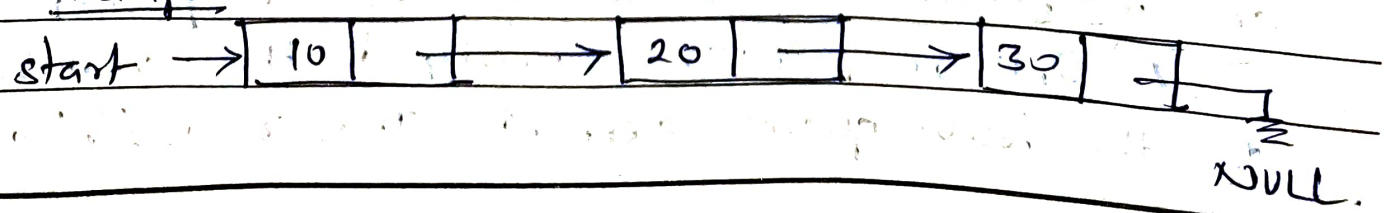
A node in a linked list usually a structure in 'C' and can be declared as follows -

```
struct node
{
    int data;
    struct node *next;
}*start = NULL;
```

- Logical representation of linked list:

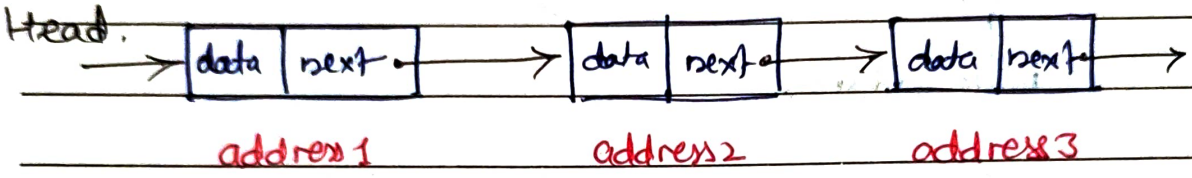


Example :-



Memory Representation of Linked List :

Unlike arrays, where elements are stored in contiguous memory locations, linked lists allocate memory dynamically for each node. This means that each node can be located anywhere in the memory and they are connected via pointers.



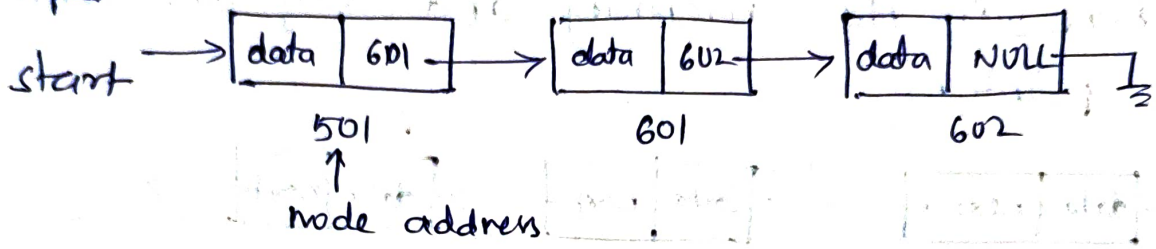
In the above linked list, each node is connected by pointers. These pointers show the address of the next node, which allowing us to move through the list in forward direction. This connection is shown with arrows in the diagram, making it clear how each node links to the next node.

Singly Linked List :-

A singly linked list is a type of data structure that is used to store a collection of elements called nodes. Each node contains a data item and a reference (address) to the next node in the sequence. The list is singly linked because each node points only to the next node in the sequence, not to the preceding one.

the first node is referred to as the "head/start" and the last node, which has NULL reference.

Example:



- Basic operations of Singly Linked List:-

① Traversal:- access each element of the linked list.

Displaying the contents of a linked list is very simple. We keep moving the 'temp' node to the next node one and display its contents. When 'temp' is 'NULL', we know that we have reached the end of the linked list so we get out of the while loop.

```
void display()
{
    struct node *q;
    if (start == NULL)
    {
        printf("List is empty");
    }
    else
```

```
q = start;
while (q != NULL)
{
    printf (" %d\t", q->data);
    q = q->next;
}
}
}
```

② Insert :-

ⓐ Insert at Beginning :-

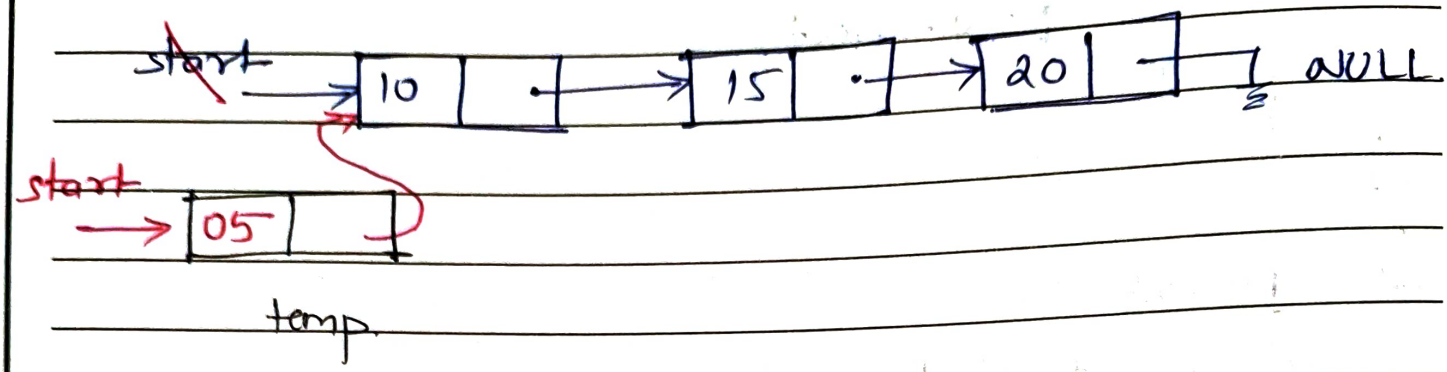
1. obtain space for new node.
2. Assign data to the data field of the new node.
3. Set the next field of the new node to the beginning of the link list.
4. change the reference pointer of the linked list to point to the new node.

memory allocation to create a node :-

```
struct node *temp;
temp = (struct node *) Malloc (sizeof (struct node));
```

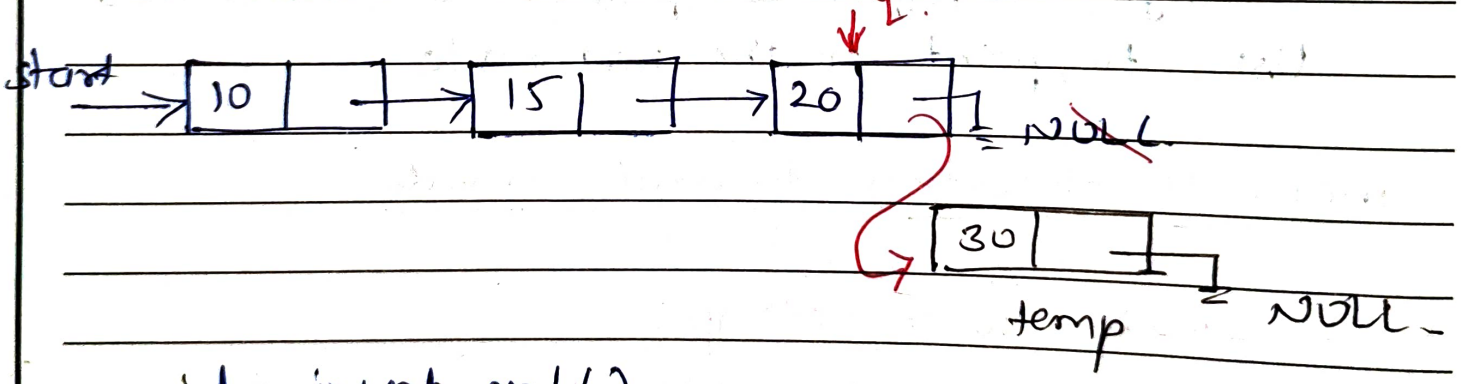
```
void insert_beg()
{ int n;
  struct node *temp;
  temp = (struct node *) malloc (sizeof (struct node));
  printf ("enter element");
  scanf ("%d", &n);
```

```
temp->data = n;
temp->next = start;
start = temp;
}
```



(b) Insert at end :-

1. Allocate memory for new node.
2. store data.
3. Traverse to last node.
4. Change next of last node to recently created node.

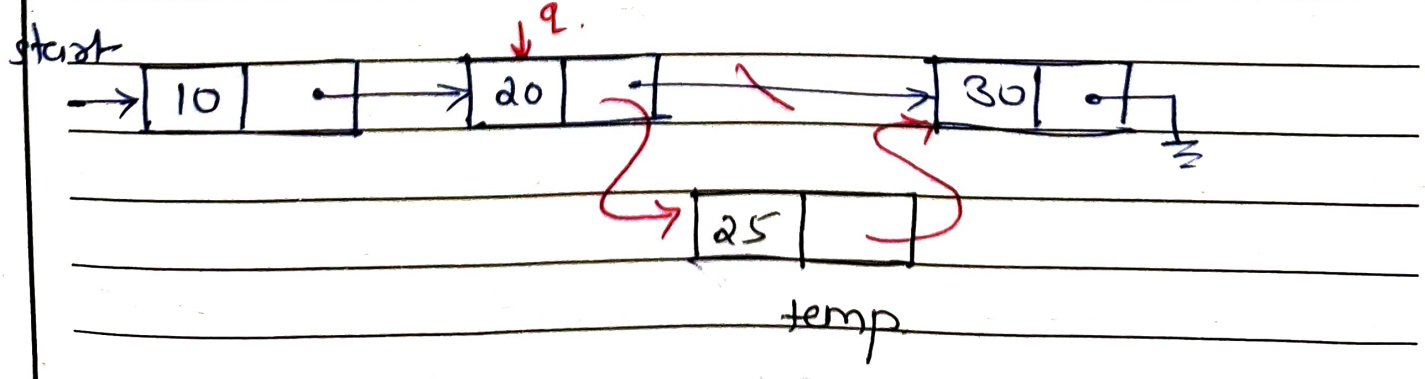



```
void insert_end()
{
    int n;
    struct node *temp, *q;
    printf("enter element");
    scanf("%d", &n);
    temp = (struct node *) malloc (sizeof(struct node));
```

```
temp→ data = n;
temp→ next = NULL;
if (start == NULL)          // if list is empty.
{
    start = temp;
}
else
{
    q = start;
    while (q→next != NULL)
    {
        q = q→next;
    }
    q→next = temp;
}
```

© Insert at mid :-

1. Allocate memory and store data for new node.
2. Traverse to node just before the required position of new node.
3. Change the next pointers to include new node in between.



Questions From DBATU Examination :

```
void addafter()
{
  int n, pos, i;
  struct node *temp, *q;
  temp = (struct node*)malloc
        (sizeof (struct node));
  printf ("enter element");
  scanf ("%d", &n);
  printf ("enter position");
  scanf ("%d", &pos);

  q = start;
  for(i=0; i<pos-1; i++)
    {
      q = q->next;
    if (q==NULL)
    {
      printf ("There are
      less than %d
      elements ", pos);
      return;
    }
  temp->data = n;
  temp->next = q->next;
  q->next = temp;
}
```

Question Bank :

+ Lecture No. : +

■ **Theory Content to be taught :**

1. singly linked list - operations on singly
2. linked list.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

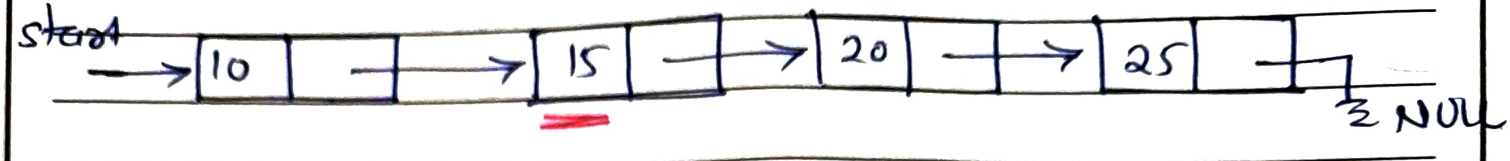
■ **Theory Content :**

③ searching a key in linked list :-

searching is performed in order to find the location of perticular element in the list. searching any element in list needs traversing through the list and make comparision of every element of the list with key element.

If the element is matched with any of list element then the location of element

is returned from function.



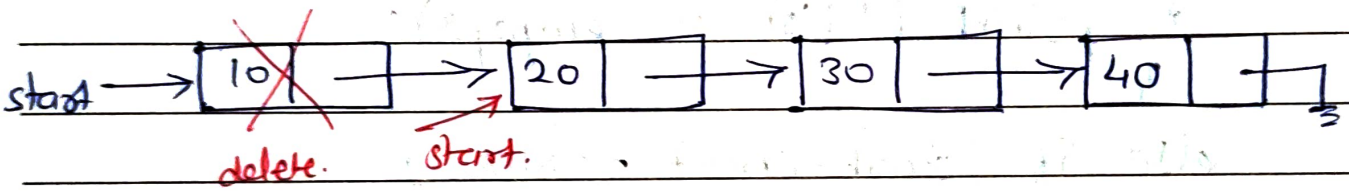
key element = 15.
element found at node 02.

```
void search()
{   int key, i=1;
    struct node *q;
    int flag=0;
    printf ("enter key element");
    scanf ("%d", &key);
    q = start;
    if (start == NULL)
    {
        printf ("List is empty");
    }
    else
    { while (q!=NULL)
      {
          if (q->data == key)
          { flag=1;
            break;
          }
          i++;
          q = q->next;
      }
    }
```

```
if (flag == 1)
{
    printf("element found at location %d", i);
}
else
{
    printf("element not found");
}
}.
```

④ **Delete**:- Removes the existing element.

ⓐ **Delete from beginning**:-

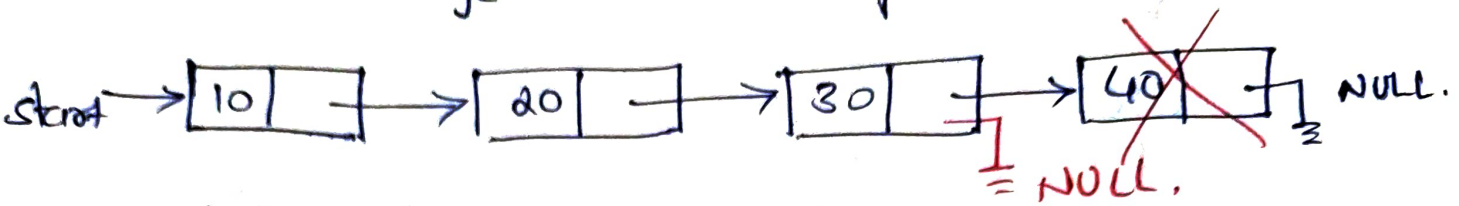


- point 'start' to the next node.

```
void del_beg()
{
    struct node *q;
    q = start;
    if (start == NULL)
    {
        printf("List is empty");
    }
    else
    {
        start = q->next;
        printf("deleted ele. is %d", q->data);
        free(q);
    }
}.
```

(b) delete from end :-

- ✓ Traverse to second last element
- ✓ change it's next pointer to null.




```
void del_end()
{
    struct node *q, *p;
    q = start;
    p = start;
    if (start == NULL)
    { printf ("List is empty");
    }
    else if (start -> next == NULL)
        {
        printf ("deleted element is %d", start->data);
        free (start);
        start = NULL)
        }
    else
      {   while (q->next != NULL)
          {
              q = q -> next)
          }
          while (p -> next != q)
            {   p = p -> next;
            }
          printf("deleted ele.is %d") q -> data);
           p -> next = NULL;
           free(q);
      }
}
```

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Linked representation of stack and Queue.
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

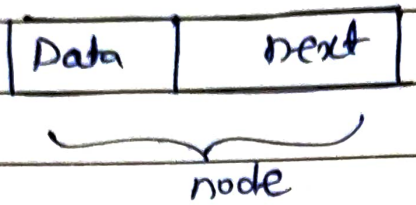
■ **Theory Content :**

● Linked representation of stack & Queue:

stack and Queue is a linear data structure that follows Last-In-first-Out (LIFO) & first-in-first-out (FIFO) principle respectively. Both can be implemented using linked list, where each element of the stack / queue is represented as a node.

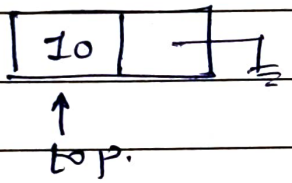
— Linked list representation of stack:

Each node contains:

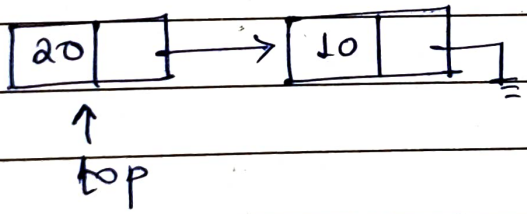


A pointer called 'top' points to the first node.

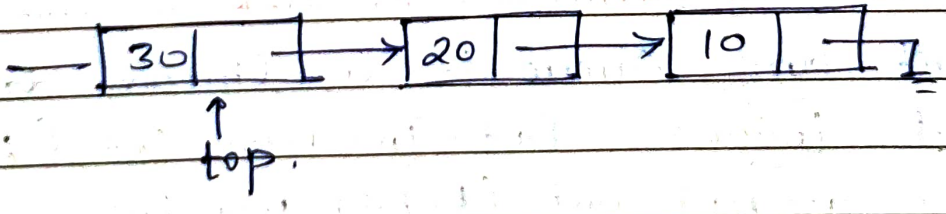
- Empty stack → top = NULL.
- After pushing 10.



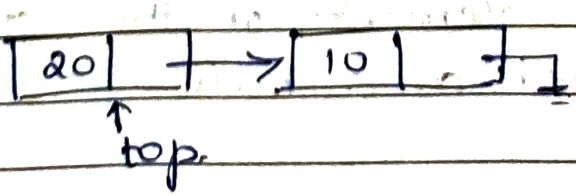
- After pushing 20.



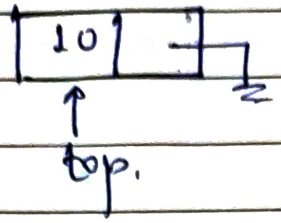
- After pushing 30.



- After calling pop operation.



- After calling pop operation.



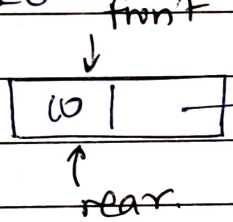
- Linked list representation of Queue :-

Queue uses two pointers.

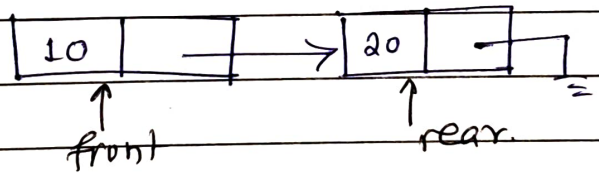
1. front 2. Rear.

- empty queue ⇒ front = rear = NULL.

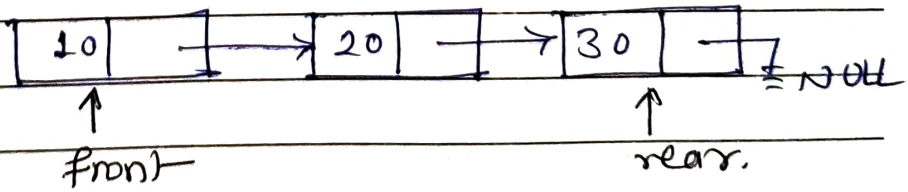
- enqueue 10.



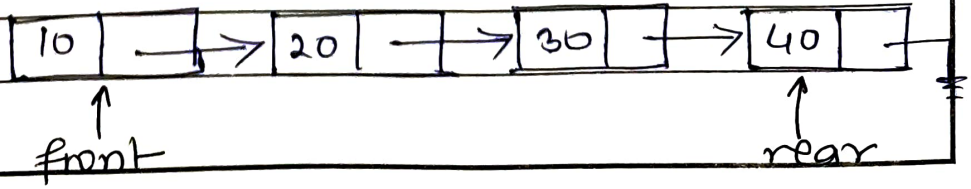
- enqueue 20.



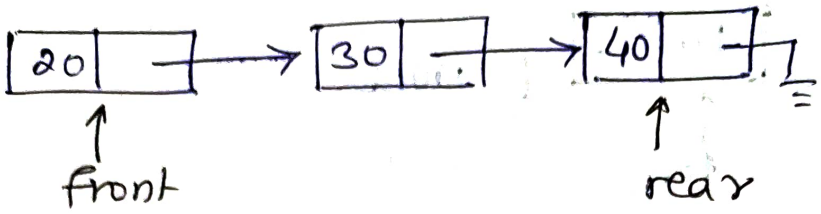
- enqueue 30



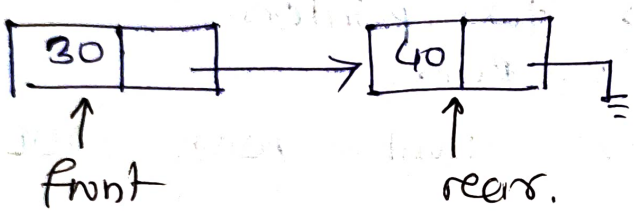
- enqueue 40



- After calling dequeue operation.



- After calling dequeue function.



+ Lecture No. : +

■ **Theory Content to be taught :**

1. Doubly linked list - operations on doubly
2. linked list.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

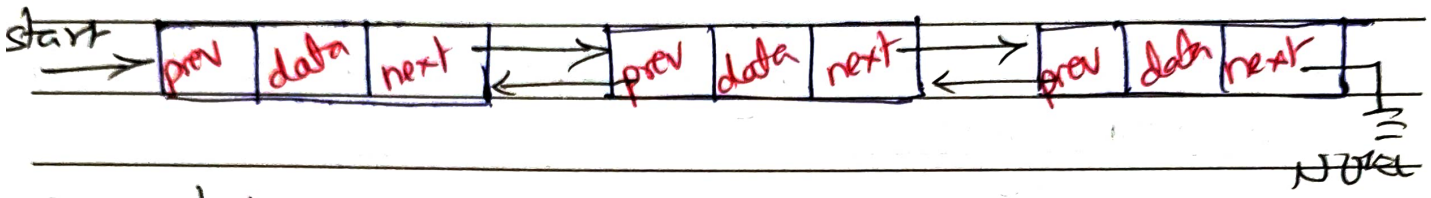
● **Doubly Linked List :-**

It is also called as two-way linked list. A linked list which can be traversed both in ~~background~~ backward as well as forward direction, is called 'Doubly linked list'.

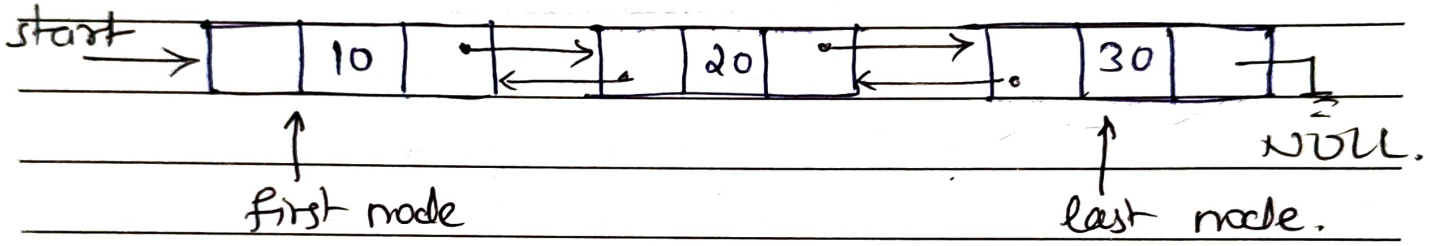
In this type of linked list each node holds two pointers field. pointer exist between adjacent nodes in both direction

In doubly linked list three fields are considered -

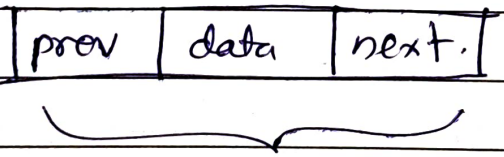
(a) reference to the previous node
(b) data field.
(c) Reference to the next node.



example :-



- Doubly linked list node :-



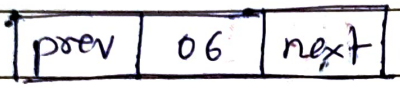

```
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
```

Operations on Doubly linked list :-

① Inserting : inserting a new node in the list

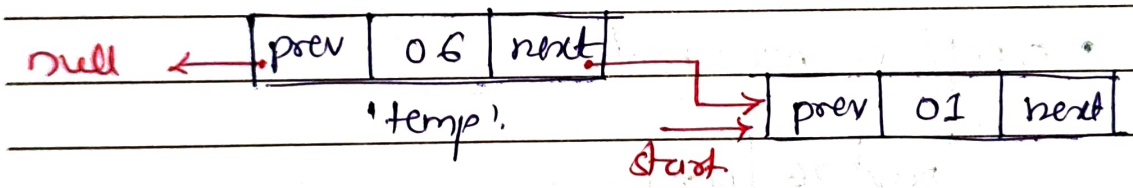
a) Inserting at Beginning :-

- create a new node.
  - ✓ allocate memory for new node 'temp'
  - ✓ assign the data to 'temp'.

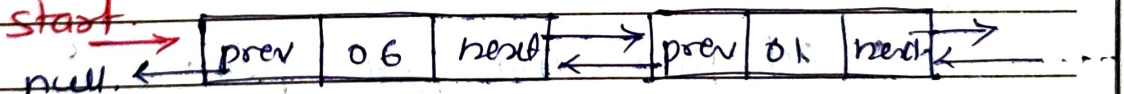


'temp'.

- set prev & next pointers of new node



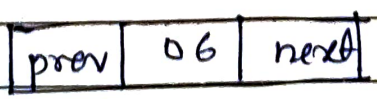
- make 'temp' node as a 'start' node.



'temp'.

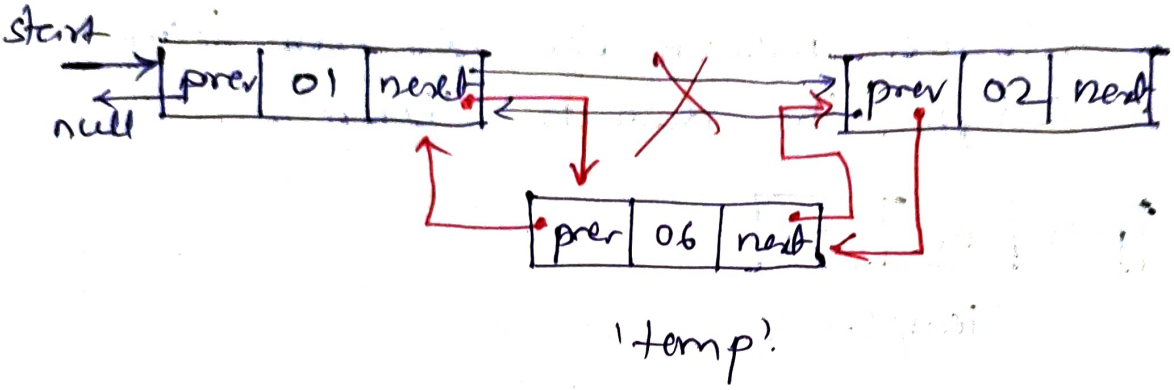
b) Insertion in between two nodes :-

- create a new node.
  - ✓ allocate memory for new node 'temp'.
  - ✓ assign the data to 'temp'.



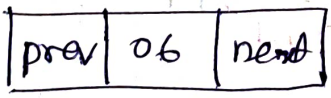
'temp'.

- set the next pointer of new node and previous node.
- set prev pointer of new node and the next node.

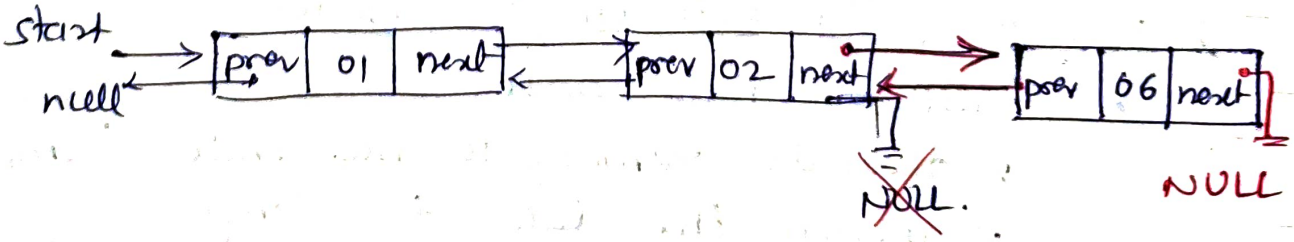


c) Insertion at end :-

- create a new node

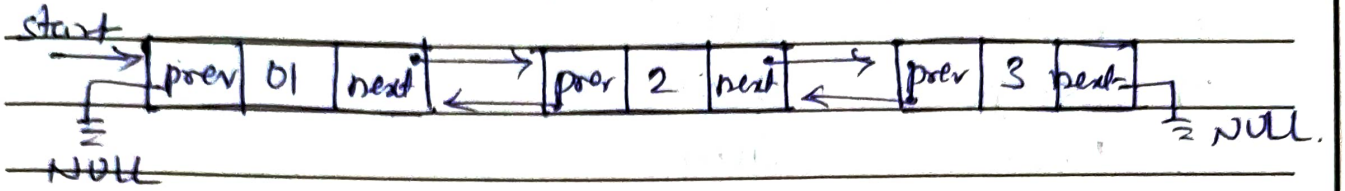


- set prev and next pointers of new node and the previous node.



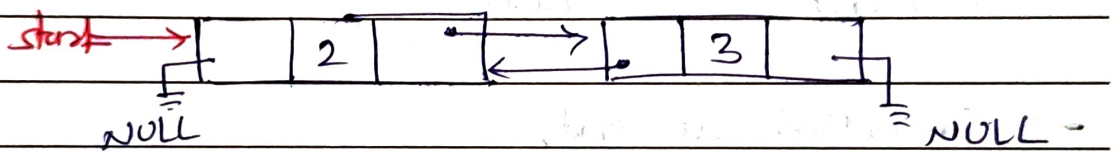
2) Deleting :- Deleting the elements from doubly linked list.

Example :- Consider following doubly linked list.



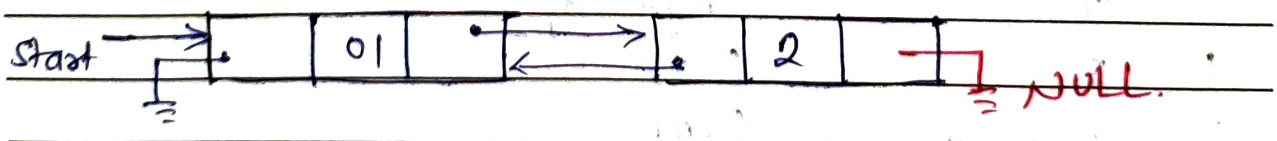
a) delete from beginning :-

Consider the second node as a 'start' and delete the first node.



b) delete from end :-

Here we can simply delete the last node, and make the 'next' of the node (previous node of the last node) point to 'NULL'.



Algorithm Analysis :-

→ Traversing a doubly linked list:

Case	Complexity
1) Best	O(n)
2) Average	O(n)
3) Worst	O(n)

→ Insertion at Beginning.

Case	Complexity
1) Best	O(1)
2) Average	O(1)
3) Worst	O(1)

→ Insertion at end.

Case	Complexity
1) Best	O(1)
2) Average	O(n)
3) Worst	O(n)

→ Insertion at specific position.

Case	Complexity
1) Best	O(1)
2) Average	O(n)
3) Worst	O(n)

→ Deletion.

	Best	average	worst
1) from beginning.	O(1)	O(1)	O(1).
2) from end.	O(1)	O(n)	O(n).
3) from specific position.	O(1)	O(n)	O(n).

→ searching in doubly linked list

case	Complexity.
1) Best	O(1)
2) ~~Best~~ average	O(n)
3) worst	O(n).

3.

4.

5.

■ **Theory Content :**

'C' program to implement Doubly Linked List

```
// struct definition.

struct node
{
    struct node *prev;
    int data;
    struct node *next;
} *start = NULL;
```

// insert at beginning :-

```
void insert_beg()
{
    struct node *temp;
    int n;
    printf ("enter a number");
    scanf ("%d", &n);
    temp=(struct node*) malloc (sizeof (struct node));

    temp→ prev = NULL;
    temp→ data = n;
    temp→ next = start;

    if (start != NULL)
        start →prev = temp;

    start = temp;
}.
```

// insert at end.

```
void insert_end()
{
    struct node *temp, *q
    int n;
    printf ("enter a number");
    scanf ("%d", &n);
    temp= (struct node *) malloc (sizeof (struct node));

        temp → data = n;
        temp → next = NULL;
```

```
if (start == NULL)
{
    temp -> prev = NULL;
    start = temp;
    return;
}
q = start;
while (q -> next != NULL)
{
    q = q -> next;
}
q -> next = temp;
temp -> prev = q;
}
```

// Display forward.

```
void display()
{
    struct node *q;
    if (start == NULL)
    {
        printf ("List is empty");
    }
    else
    {
        while (q != NULL)
        { printf ("%d <-> ", q -> data);
          q = q -> next;
        } }
    printf ("NULL");
}
```

// Delete from beginning.

```
void delete_beg()
{
    struct node *q;
    if (start == NULL)
    {
        printf("List is empty");
        return;
    }
    q = start;
    start = start->next;
    if (start != NULL)
        start->prev = NULL;
    printf("deleted element is : %d", q->data);
    free(q);
}
```

// Delete from end.

```
void delete_end()
{
    struct node *q, *p;
    if (start == NULL)
    {
        printf("List is empty");
        return;
    }
```

```
q = start;
while ( q -> next != NULL)
{
        p = q;
        q = q -> next;
}
if (q == start)
{
        start = NULL;
}
else
{
        p -> next = NULL;
}
free(q);
}.
```

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Circular linked list : all operations their
2. algorithms and the complexity analysis
3.
4.
5.

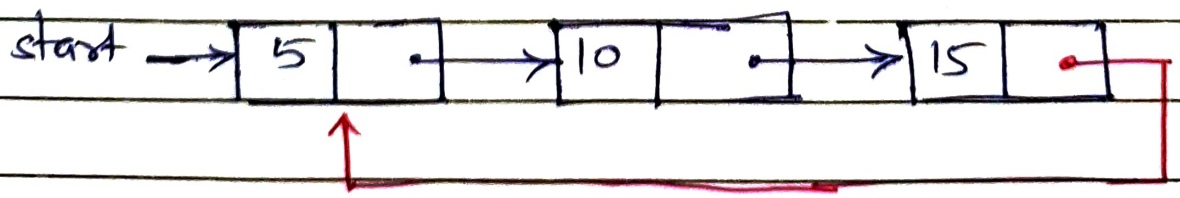
■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

① Circular linked list :-

A circular linked list is a linked list in which the last node points back to the first node (start), forming a circle.



Characteristics :-

1) No null pointer at the end.
2) Traversal can start from any node

- Node structure :-

```
struct node
{
    int data;
    struct node *next;
};
```

- Operations on Circular Linked List :-

① Traversal operation :-

1. Check if list is empty.
2. Start from 'start' node.
3. display data.
4. Move the next node
5. Repeat until current node becomes 'start' node again.

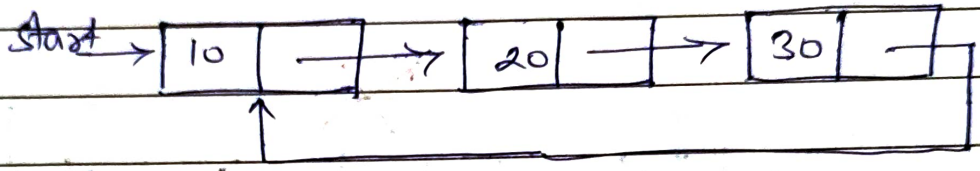
- Time Complexity = O(n).

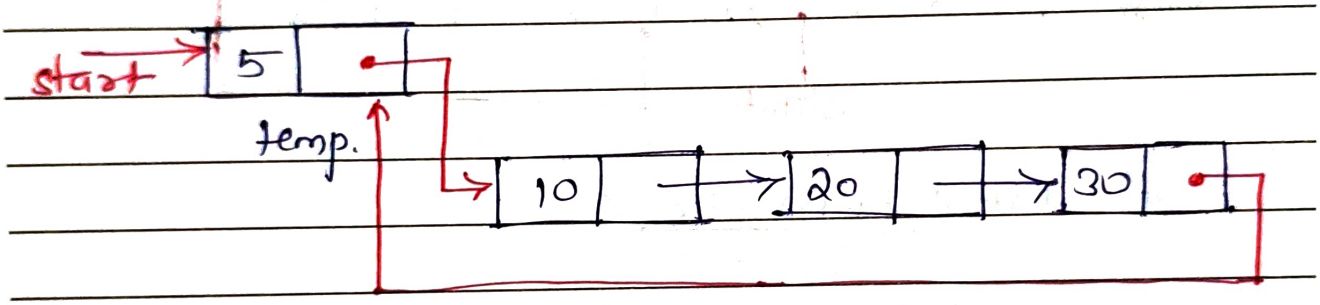
② Insertion :-

(a) Insertion at Beginning :-

1. Create new node
2. Insert node before 'start' node
3. Last node's 'next' should point to new node.
4. Update 'start'.

Ex :-

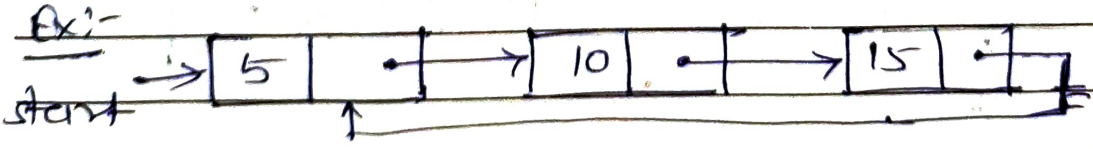


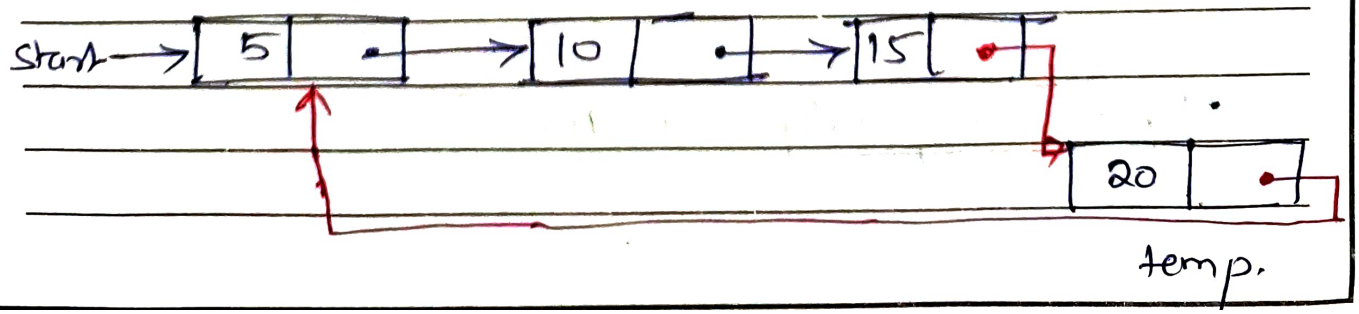


(b) insertion at end :-

1. Create a new node
2. find Last node
3. Last node's next = temp, i.e. new node
4. New node's (start) next = start.

Ex :-



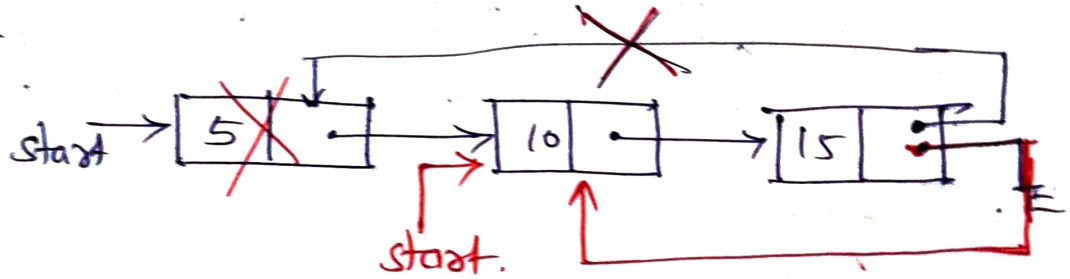


③ Deletion :-

(a) Deletion from beginning :-

1. find Last node
2. Last node's next = second node.
3. move start to second node.
4. Delete first node.

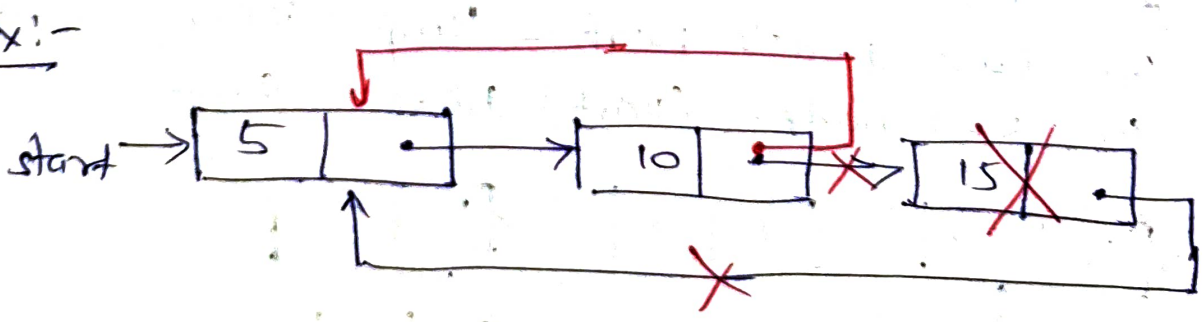
Ex :-



(b) delete from end :-

1. Traverse to last node
2. find previous node.
2. previous node points to start
4. delete Last node.

Ex :-



Time Complexity = O(n).